Guia completa de

Agent Reach

Instalacion, configuracion, uso, actualizacion, resolucion de problemas y seguridad, paso a paso

`Repositorio: github.com/Panniantong/Agent-Reach`

Fuentes: README (chino e ingles), docs/install.md, docs/update.md y docs/troubleshooting.md del repositorio, consultados el 29 de septiembre de 2026. El proyecto cambia con frecuencia: ante cualquier diferencia, manda lo que diga el repositorio.

Indice

1. Que es Agent Reach
2. Plataformas soportadas
3. Requisitos previos
4. Solo si usas OpenClaw: activar el permiso exec
5. Instalacion, opcion A: que lo haga tu agente
6. Instalacion, opcion B: manual paso a paso
7. Elegir canales opcionales
8. Verificacion con doctor
9. Configuracion de cada canal
10. Uso diario
11. Actualizar
12. Monitorizacion diaria (OpenClaw)
13. Resolucion de problemas
14. Seguridad y privacidad
15. Desinstalar
16. Referencia rapida de comandos
17. Enlaces y creditos

1. Que es Agent Reach

Agent Reach es una **capa de capacidades** para agentes de IA. No es un wrapper ni otra herramienta de lectura: se encarga de **elegir, instalar, diagnosticar y enrutar** las herramientas upstream que dan acceso a internet, y despues tu agente las llama directamente (`twitter`, `yt-dlp`, `gh`, `opencli`, `bili`, `mcporter`...).

- Licencia MIT. Python 3.10 o superior. Alrededor de 80.6k estrellas en GitHub.
- **Gratis**: todas las herramientas son de codigo abierto y las APIs que usa son gratuitas. El unico coste posible es un proxy de servidor (aprox. 1 USD al mes), y solo si lo despliegas en un servidor. En un ordenador local no hace falta proxy.
- Compatible con cualquier agente que pueda ejecutar comandos: Claude Code, OpenClaw, Cursor, Windsurf...
- Cada plataforma tiene una **lista ordenada de backend preferido mas alternativos**. Si un acceso deja de funcionar se pasa al siguiente. Ejemplo del README: en junio de 2026 Bilibili bloqueo yt-dlp (error 412) y el proyecto cambio a bili-cli sin que el usuario hiciera nada.
- Diagnostico integrado: `agent-reach doctor` indica que canal funciona, cual no y como arreglarlo.
- Las cookies se guardan solo en tu maquina; no se suben ni se comparten.

> **Aviso importante:** La version inglesa del README avisa de que Agent Reach **no tiene ningun token, moneda, producto de inversion, programa de comisiones ni proyecto de Solana/Pump.fun**. Cualquier proyecto cripto que use el nombre, la URL de GitHub o la identidad del autor no esta afiliado al repositorio. No conectes una wallet ni reclames comisiones por mensajes o enlaces que digan lo contrario.

Por que existe

Un agente ya puede programar o editar documentos, pero al buscar informacion fuera se encuentra barreras: subtitulos de YouTube inaccesibles, la API de Twitter de pago (el README cita unos 215 USD al mes con uso moderado), Reddit devolviendo 403 a IPs de servidor, Xiaohongshu exigiendo login, Bilibili bloqueando herramientas genericas, paginas web que llegan como HTML sin limpiar, y los feeds RSS que exigen instalar librerias. Configurarlo todo a mano, plataforma por plataforma, es lento. Agent Reach lo reduce a una frase que le pasas a tu agente.

2. Plataformas soportadas

Plataforma	Funciona al instalar	Se desbloquea al configurar	Como se configura
Web	Leer cualquier pagina (Jina Reader)	-	Nada
YouTube	Subtitulos + busqueda (yt-dlp)	-	Nada
RSS	Leer feeds RSS/Atom (feedparser)	-	Nada
Busqueda web	-	Busqueda semantica (Exa via mcporter)	Automatico al instalar, gratis y sin clave
GitHub	Repos publicos + busqueda	Repos privados, Issues/PR, Fork	`gh auth login` o pedirselo al agente
Twitter/X	Leer un tuit	Buscar, timeline, articulos largos	Cookies con Cookie-Editor
Bilibili	Busqueda + detalle de video (bili-cli, sin login)	Subtitulos (OpenCLI)	Pedirselo al agente

Plataforma	Funciona al instalar	Se desbloquea al configurar	Como se configura
Reddit	Nada: no hay ruta sin configuracion	Busqueda + posts y comentarios	OpenCLI (sesion del navegador) o rdt-cli + cookie
Facebook	-	Busqueda, perfiles, feed, lista de grupos	OpenCLI (sesion de Chrome)
Instagram	-	Busqueda de usuarios, perfil, ultimos posts, Explore	OpenCLI (sesion de Chrome)
Xiaohongshu	-	Busqueda, lectura, comentarios	OpenCLI (sesion de Chrome existente) o MCP / xhs-cli con Cookie-Editor
LinkedIn	Paginas publicas via Jina Reader	Perfiles, empresas, busqueda de empleos	mcp-server-linkedin
V2EX	Temas populares, por nodo, detalle + respuestas, usuarios	-	Nada
Xueqiu	-	Cotizaciones, busqueda, posts y acciones populares	Cookie del navegador
Podcast Xiaoyuzhou	-	Audio a texto (Whisper via Groq)	Clave gratuita de Groq
Boss Zhipin	-	Busqueda de ofertas + descripcion completa	Solo escritorio, via CDP (seccion 9.10)

Para contenido tecnico en espanol, los canales mas utiles suelen ser: web, YouTube, GitHub, RSS, busqueda Exa, Twitter/X y Reddit. El resto esta orientado sobre todo a plataformas chinas.

Backends por plataforma (orden de preferencia)

```
channels/
  web.py          Jina Reader
  twitter.py      twitter-cli > OpenCLI > bird
  youtube.py      yt-dlp
  github.py       gh CLI
  bilibili.py     bili-cli > OpenCLI > API de busqueda   (yt-dlp retirado: bloqueado por Bilibili)
  reddit.py       OpenCLI > rdt-cli                     (sin ruta sin configuracion)
  facebook.py     OpenCLI                               (sesion del navegador de escritorio)
  instagram.py    OpenCLI                               (sesion del navegador de escritorio)
  xiaohongshu.py  OpenCLI > xiaohongshu-mcp > xhs-cli
  linkedin.py     mcp-server-linkedin > Jina Reader
  rss.py          feedparser
  exa_search.py   Exa via mcporter
  __init__.py     registro de canales (lo usa doctor)
```

Cada archivo de canal **prueba de verdad** sus backends candidatos en orden (no solo comprueba que el comando exista). El primero que funciona por completo pasa a ser el backend activo; si uno esta roto, doctor da la receta para repararlo.

Escenario	Preferido	Alternativo	Motivo
Leer paginas web	Jina Reader	-	Gratis, sin clave
Leer Twitter	twitter-cli	OpenCLI	Busqueda estable en pruebas reales; OpenCLI usa la sesion del navegador
Reddit	OpenCLI (escritorio)	rdt-cli	Rutas anonimas bloqueadas y API oficial con aprobacion: solo queda la sesion iniciada
Facebook	OpenCLI (escritorio)	-	Graph API y Groups API muy restringidas
Instagram	OpenCLI (escritorio)	API Graph oficial (Business/Creator + revision)	Las vias tipo instaloader son inestables

Escenario	Preferido	Alternativo	Motivo
YouTube	yt-dlp	-	Sigue siendo lo mejor para YouTube (ya no se usa para Bilibili)
Bilibili	bili-cli	OpenCLI, API de busqueda	yt-dlp recibe 412; bili-cli busca y lee sin login
Buscar en la web	Exa via mcporter	-	Busqueda semantica con IA, via MCP y sin clave
GitHub	gh CLI	-	Herramienta oficial; API completa tras autenticar
RSS	feedparser	-	Estandar del ecosistema Python
Xiaohongshu	OpenCLI (escritorio)	xiaohongshu-mcp (servidor), xhs-cli	OpenCLI solo usa una sesion existente del usuario
LinkedIn	mcp-server-linkedin	Jina Reader	Servicio MCP con automatizacion de navegador

Son las opciones *actuales*, revisadas periodicamente en maquinas reales. `agent-reach doctor` siempre dice cual esta activa.

3. Requisitos previos

- **Python 3.10 o superior.**
- **pipx** (recomendado) o la posibilidad de crear un entorno virtual.
- **Node.js** (el instalador lo comprueba; hace falta para mcporter y OpenCLI).
- **gh CLI** (el instalador lo comprueba).
- **uv / uvx**, solo si vas a configurar LinkedIn.
- **Google Chrome**, solo si usas OpenCLI (Reddit, Facebook, Instagram, Xiaohongshu, subtitulos de Bilibili, respaldo de Twitter).
- Si usas **OpenClaw**, activar antes el permiso de ejecucion (seccion 4).
- Muy recomendable: **cuentas secundarias** para todo lo que requiera cookies o sesion de navegador (seccion 14).

4. Solo si usas OpenClaw: activar el permiso exec

Agent Reach necesita que el agente ejecute comandos de shell (`pip install`, `mcporter`, `twitter`...). Con el perfil de herramientas por defecto de OpenClaw (`messaging`) el agente no puede. Antes de instalar:

```
openclaw config set tools.profile "coding"
```

Alternativa: editar `~/.openclaw/openclaw.json` y poner `"tools": { "profile": "coding" }`. Despues reinicia el Gateway y abre una conversacion nueva:

```
openclaw gateway restart
```

Claude Code, Cursor, Windsurf y similares no tienen esta limitacion.

5. Instalacion, opcion A: que lo haga tu agente

Es el metodo que recomienda el repositorio. Copia esta frase y pegala en tu agente (Claude Code, OpenClaw, Cursor...):

```
Instala Agent Reach: https://raw.githubusercontent.com/Panniantong/agent-reach/main/docs/install.md
```

El README original usa la version en chino de la frase; el agente lee el mismo documento, asi que la version en espanol o ingles funciona igual. Para una instalacion **segura por defecto** (solo comprueba, no modifica nada):

```
Revisa de forma segura e instala Agent Reach: https://raw.githubusercontent.com/Panniantong/agent-reach/main/docs/ins
```

Que hace el agente, en orden

1. Instala el CLI `agent-reach` desde este repositorio (incluye yt-dlp y feedparser). **No lo instales desde PyPI**: el paquete con el mismo nombre no es este proyecto.
2. Comprueba Node.js, gh CLI y mcporter, y te indica como instalar lo que falte.
3. Solo si se pasa `--system` de forma explicita, instala dependencias y conecta Exa por MCP.
4. Detecta si estas en un ordenador local o en un servidor y ajusta las recomendaciones.
5. Solo con `--system`, registra el `SKILL.md` en el directorio de skills de tu agente. Sin ese flag no toca archivos.
6. Te pregunta que canales opcionales quieres. Por defecto solo activa los canales sin configuracion; los que necesitan sesion (Xiaohongshu, Twitter, Reddit, Facebook, Instagram) se instalan cuando los pides por su nombre.

Reglas que el propio instalador impone al agente

- No usar `sudo` salvo autorizacion explicita del usuario.
- No modificar archivos del sistema fuera de `~/.agent-reach/`.
- No instalar paquetes que no figuren en la guia.
- No desactivar cortafuegos ni protecciones del sistema.
- No clonar repositorios, crear archivos ni ejecutar comandos dentro del directorio de trabajo del agente.
- Si algo requiere permisos elevados, avisar al usuario y dejar que decida.

Uso	Directorio	Ejemplo
Configuracion y tokens	`~/.agent-reach/`	`~/.agent-reach/config.yaml`
Repos de herramientas upstream	`~/.agent-reach/tools/`	`~/.agent-reach/tools/xiaoyuzhou/`
Temporales	`/tmp/`	`/tmp/yt-dlp-output/`
Skills (OpenClaw)	`~/.openclaw/skills/agent-reach/`	SKILL.md

Motivo: si se clonan repos o se crean archivos en el espacio de trabajo, se ensucia el proyecto del usuario y puede romper el agente con el tiempo.

6. Instalacion, opcion B: manual paso a paso

6.1 Con pipx (lo mas comodo)

```
pipx install https://github.com/Panniantong/agent-reach/archive/main.zip
agent-reach install --env=auto
```

6.2 Con entorno virtual

Usalo si tu Python viene de Homebrew o si ves el error `externally-managed-environment` (proteccion PEP 668 del sistema, no un fallo de Agent Reach).

```
python3 -m venv ~/.agent-reach-venv
source ~/.agent-reach-venv/bin/activate
pip install https://github.com/Panniantong/agent-reach/archive/main.zip
agent-reach install --env=auto
```

6.3 Windows

Si `python3 --version` abre la Microsoft Store, o `where python3` apunta a `...\AppData\Local\Microsoft\WindowsApps\python3.exe`, es un alias de la Store y no un Python real. Usa el lanzador `py -3` o el `python.exe` de la carpeta de instalacion. En PowerShell:

```
py -3 -m venv $env:USERPROFILE\.agent-reach-venv
$env:USERPROFILE\.agent-reach-venv\Scripts\Activate.ps1
python -m pip install https://github.com/Panniantong/agent-reach/archive/main.zip
agent-reach install --env=auto
```

6.4 Como Skill (Claude Code, OpenClaw o cualquier agente con skills)

```
npx skills add Panniantong/Agent-Reach@agent-reach
```

El agente detecta si el CLI `agent-reach` esta disponible y lo instala si hace falta. Si instalas herramientas externas con `agent-reach install --system`, la skill se registra automaticamente; la comprobacion de solo lectura no toca archivos existentes.

Si prefieres la skill solo en ingles, define un locale ingles o exporta `AGENT_REACH_LANG=en` antes de ejecutar `agent-reach install --env=auto --system` o `agent-reach skill --install`. El archivo siempre se escribe como `SKILL.md`: para cambiar de idioma hay que repetir el comando con el nuevo locale, y se reemplaza el archivo anterior.

6.5 Modos del comando install

Comando	Que hace
`agent-reach install --env=auto`	Comprobacion de solo lectura; lista lo que falta. Es el modo por defecto.
`agent-reach install --env=auto --safe`	Igual que el anterior (parametro de compatibilidad).
`agent-reach install --env=auto --dry-run`	Previsualiza lo que haria --system, sin cambiar nada.
`agent-reach install --env=auto --system`	Instala y configura herramientas externas. Solo cuando aceptes que se modifique tu maquina.

Recomendacion: ejecuta primero `--dry-run`, revisa la salida y despues `--system`.

6.6 Que se activa con --system

Canales sin configuracion: Web (Jina Reader), YouTube, GitHub, RSS, Exa Search, V2EX y Bilibili (basico). Ademas se comprueban o configuran gh CLI, Node.js, mcporter, Exa y la configuracion de yt-dlp.

7. Elegir canales opcionales

Tras lo basico, el agente te ofrece esta lista y solo instala lo que pidas:

- **OpenCLI** (recomendado en escritorio): una instalacion cubre Reddit, Facebook, Instagram, subtitulos de Bilibili, respaldo de Twitter y backend de escritorio de Xiaohongshu.
- **Twitter/X**: buscar tuits y ver timelines (requiere cookies).

- **Xueqiu**: cotizaciones y posts (requiere cookies).
- **Podcast Xiaoyuzhou**: audio a texto (requiere clave gratuita de Groq).
- **Xiaohongshu**: busqueda, lectura y comentarios.
- **Reddit**: busqueda y lectura (requiere sesion: OpenCLI o rdt-cli + cookie).
- **Facebook** e **Instagram** (escritorio, via OpenCLI).
- **Bilibili completo** (bili-cli, sin login), **LinkedIn** y **Boss Zhipin**.

Comandos de ejemplo:

```
agent-reach install --env=auto --system --channels=opencli,xiaohongshu   # escritorio + Xiaohongshu
agent-reach install --env=auto --system --channels=facebook,instagram    # Meta en escritorio
agent-reach install --env=local --system --channels=boss                  # Boss Zhipin, escritorio
agent-reach install --env=auto --system --channels=all                    # todo, con tu aprobacion
```

Nombres validos de canal: `opencli, twitter, xiaoyuzhou, xueqiu, xiaohongshu, reddit, facebook, instagram, bilibili, linkedin, boss, all`.

8. Verificacion con doctor

```
agent-reach doctor
agent-reach doctor --json     # salida para maquinas; incluye active_backend por canal
agent-reach version
```

Muestra cada canal como listo, parcial o roto, con el mensaje de como arreglarlo. En las plataformas con varios backends (Xiaohongshu, Reddit, Bilibili, Twitter) indica el backend activo. Si un canal que funcionaba ahora falla, el mensaje incluye el arreglo exacto (por ejemplo, una receta de reinstalacion en venv).

En modo texto, `doctor` tambien se asegura de que exista una skill de Agent Reach en los directorios de skills detectados. Si ya tienes una, la respeta y no la sobrescribe. Para refrescar los archivos de la skill incluida:

```
agent-reach skill --install
```

Ejemplo de salida (del README)

```
$ agent-reach doctor

Agent Reach Status
========================================
Ready to use:
  OK  GitHub repos and code - public repos readable and searchable
  OK  YouTube video subtitles - yt-dlp
  OK  Bilibili search & video detail - bili-cli (subtitles via OpenCLI)
  OK  RSS/Atom feeds - feedparser
  OK  Web pages (any URL) - Jina Reader API

Search (free Exa key to unlock):
  --  Web semantic search - sign up at exa.ai for free key

Configurable:
  !!  Twitter/X - doctor only checks that explicit credentials exist
  --  Reddit posts and comments - needs login: rdt-cli after `rdt login`, or OpenCLI
  --  XiaoHongShu notes - OpenCLI needs an existing user-controlled session
  --  Facebook / Instagram - desktop: OpenCLI browser session

Status: 6/9 channels available
```

(Los iconos originales del README se han sustituido por texto simple.)

9. Configuracion de cada canal

Regla general del repositorio: no hace falta consultar documentacion. Puedes decirle a tu agente "configurame X" y te guia paso a paso. Aun asi, esto es lo que ocurre en cada caso.

9.1 Cookies con Cookie-Editor (metodo preferido para Twitter, Xueqiu, etc.)

1. Inicia sesion en la plataforma en tu navegador, con una **cuenta secundaria**.
2. Instala la extension **Cookie-Editor** para Chrome.
3. Pulsa la extension, luego **Export** y despues **Header String**.
4. Pega el texto exportado a tu agente.

9.2 Twitter/X

```
agent-reach configure twitter-cookies
```

Esto guarda `twitter_auth_token` y `twitter_ct0` **solo para que doctor compruebe que la configuracion esta completa**. No ejecuta `twitter status` ni modifica tu shell actual. Para usar el comando `twitter` directamente, hay que definir las variables en el proceso que lo lanza:

```
export TWITTER_AUTH_TOKEN="..."
export TWITTER_CT0="..."
twitter search "query" -n 10
```

Backends por orden: twitter-cli, luego OpenCLI, luego bird (`npm install -g @steipete/bird`, si ya lo tienes instalado se detecta automaticamente).

9.3 Proxy (solo redes restringidas o servidores)

La mayoria de usuarios no necesita proxy. Si tu red bloquea Reddit o Twitter (por ejemplo, China continental) o si tu servidor tiene la IP bloqueada, el README recomienda un proxy residencial (Webshare, aprox. 1 USD al mes). Le pasas la direccion al agente, la guarda y exporta las variables al llamar a esas herramientas.

```
agent-reach configure proxy      # entrada oculta
export HTTP_PROXY="..." HTTPS_PROXY="..."
```

twitter-cli y rdt-cli usan Python y respetan esas variables. Reddit requiere sesion iniciada de todos modos. Bilibili funciona con bili-cli sin proxy.

9.4 Reddit (el login es obligatorio)

Las rutas anonimas estan bloqueadas y la API oficial exige aprobacion manual. Opciones:

- **Escritorio:** OpenCLI, con sesion iniciada en reddit.com en tu navegador.
- **Servidor o instalacion previa:** rdt-cli, instalado desde GitHub (PyPI va por detras) con esta version fijada por commit:

```
pipx install \
  'git+https://github.com/public-clis/rdt-cli.git@5e4fb3720d5c174e976cd425ccc3b879d52cac66'
rdt login     # extrae la cookie del navegador; sin navegador, sigue lo que indique doctor
```

Uso: `opencli reddit search "query" -f yaml` o `rdt read POST_ID`.

9.5 OpenCLI (base de varios canales)

```
agent-reach install --system --channels opencli
```

Queda un paso manual que nadie puede hacer por ti (limitacion de seguridad de Chrome):

1. Abre `https://chromewebstore.google.com/detail/opencli/ildkmabpimmkaediidaifkhjpohdnifk`
2. Pulsa **Anadir a Chrome**.
3. Ejecuta `opencli doctor` y comprueba que muestra `Extension: connected`.

9.6 Facebook e Instagram (escritorio, via OpenCLI)

Reutilizan tu sesion de Chrome: no guardan usuario ni contrasena y no usan la aprobacion de la Graph API de Meta. No se recomiendan en servidores o entornos sin escritorio.

```
agent-reach install --system --channels facebook,instagram
```

1. Comprueba que Chrome tiene la extension OpenCLI y que `opencli doctor` da resultado correcto.
2. Inicia sesion en facebook.com e instagram.com en Chrome.
3. El agente ya puede usar estos comandos:

```
opencli facebook search "query" -f yaml
opencli facebook profile zuck -f yaml
opencli facebook groups -f yaml
opencli instagram search "query" -f yaml      # busqueda de USUARIOS
opencli instagram profile nasa -f yaml
opencli instagram user nasa -f yaml           # ultimos posts de un usuario
```

> **Limitaciones:** Facebook Groups solo garantiza leer la lista de grupos y la actividad reciente visibles tras tu login; no hay API de posts y comentarios arbitrarios de grupos. La busqueda de Instagram es de usuarios, no de posts por palabra clave. Si aparece un error 429 o de login: vuelve a iniciar sesion en Chrome y baja la frecuencia de peticiones.

9.7 Xiaohongshu (varios backends)

Agent Reach **no inicia sesion por ti ni lee las cookies del navegador** en Xiaohongshu. OpenCLI solo usa una sesion de Chrome que ya exista y que controles tu. El comando `agent-reach configure xhs-cookies` no inyecta cookies en OpenCLI ni en Chrome. Si no tienes una sesion, no se automatiza el login: se exporta manualmente con Cookie-Editor y se configura xiaohongshu-mcp o una herramienta heredada:

```
agent-reach configure xhs-cookies
```

Ese comando guarda las cookies que aportes del dominio xiaohongshu.com (las de otros dominios se ignoran). Revisa los nombres y el alcance antes de usarlo.

Escritorio (recomendado): OpenCLI

Igual que en 9.5. Uso: `opencli xiaohongshu search "query" -f yaml`. Si aparece `AUTH_REQUIRED` y no tienes sesion, no se hace login automatico: usa la ruta de servidor siguiente con Cookie-Editor.

Servidor o sin escritorio: xiaohongshu-mcp

1. Descarga el binario para tu plataforma desde `https://github.com/xpzouying/xiaohongshu-mcp/releases` a `~/.agent-reach/tools/`.
2. Arranca el servicio. La primera vez descarga un navegador headless de unos 150 MB: espera a que termine.
3. Importa las cookies manualmente con Cookie-Editor (9.1).
4. Conectalo: `mcporter config add xiaohongshu http://localhost:18060/mcp --scope home`
5. Al llamarlo, anade siempre `--timeout 120000`.

Usuarios existentes: xhs-cli

Si ya lo tenias instalado sigue funcionando como alternativa. El proyecto upstream esta sin mantenimiento desde marzo de 2026, asi que no se recomienda para instalaciones nuevas. La autenticacion sigue siendo el flujo manual con Cookie-Editor.

9.8 Xueqiu

Inicia sesion en xueqiu.com en Chrome y despues ejecuta:

```
agent-reach configure --from-browser chrome --platform xueqiu
```

Solo lee y guarda las cookies minimas necesarias de Xueqiu, no las de otras plataformas. Cuando caduquen, repite el comando.

9.9 Podcast Xiaoyuzhou (transcripcion con Groq Whisper)

El script se instala con Agent Reach. Solo falta la clave gratuita:

```
agent-reach configure groq-key
```

Como conseguir la clave (gratis, sin tarjeta, unos 30 segundos):

1. Abre `https://console.groq.com`
2. Entra con tu cuenta de Google o GitHub (o registrate).
3. En el menu izquierdo: **API Keys**, luego **Create API Key**.
4. Copia la clave (empieza por `gsk_`) y pasasela al agente.

Uso: envia un enlace de episodio al agente, o ejecuta:

```
bash ~/.agent-reach/tools/xiaoyuzhou/transcribe.sh https://www.xiaoyuzhoufm.com/episode/xxxxx
```

El proceso descarga el audio, lo transcodifica y trocea, lo transcribe con Whisper en Groq y devuelve el texto completo.

- Limite gratuito: aproximadamente 2 horas de audio por hora (7200 segundos). Al excederlo, espera 15 minutos.
- Calidad alta (Whisper large-v3), pero **no distingue hablantes**.
- Para podcasts de mas de 2 horas, procesalos por partes.

9.10 LinkedIn (mcp-server-linkedin)

Lo basico se lee con Jina Reader. Para perfiles completos, empresas y busqueda de empleos hace falta mcp-server-linkedin.

1. Instala **uv** siguiendo `https://docs.astral.sh/uv/getting-started/installation/` (incluye `uvx`).
2. Registra el servicio (modo recomendado, stdio):

```
mcporter config add linkedin --command uvx --arg mcp-server-linkedin@latest \
  --env UV_HTTP_TIMEOUT=300 --scope home
```

Primer login (necesita interfaz grafica):

```
uvx mcp-server-linkedin@latest --login
```

Se abre un navegador donde inicias sesion manualmente en LinkedIn; el estado se guarda en `~/.linkedin-mcp/profile/`. En un servidor sin escritorio hay que ejecutar ese mismo comando desde un escritorio visible (por ejemplo, VNC). Ejemplo de uso: `mcporter call linkedin.get_person_profile`

`linkedin_username="..."`. Mas detalles: github.com/stickerdaniel/linkedin-mcp-server. `uvx` obtiene y arranca la ultima version cuando hace falta, sin instalar paquetes Python ni mantener un servicio HTTP.

9.11 Boss Zhipin (solo escritorio, boss-agent-cli + CDP)

Agent Reach automatiza lo que se puede automatizar; solo el login en la web queda para ti.

1. El agente te explica que va a instalar una CLI upstream, lanzar un Chrome con perfil independiente y pedir autorizacion de sistema.
2. Con tu permiso: `agent-reach install --env=local --system --channels=boss`
3. Lanza el Chrome dedicado, **solo en la interfaz local (127.0.0.1)**:

```
# macOS
open -na "Google Chrome" --args --remote-debugging-address=127.0.0.1 \
  --remote-debugging-port=9222 --user-data-dir="$HOME/.boss-chrome-profile" \
  "https://www.zhipin.com/web/geek/job"

# Linux
google-chrome --remote-debugging-address=127.0.0.1 \
  --remote-debugging-port=9222 --user-data-dir="$HOME/.boss-chrome-profile" \
  "https://www.zhipin.com/web/geek/job"

# Windows PowerShell
Start-Process chrome.exe -ArgumentList '--remote-debugging-address=127.0.0.1',
  '--remote-debugging-port=9222',
  "--user-data-dir=$env:USERPROFILE\.boss-chrome-profile",
  'https://www.zhipin.com/web/geek/job'
```

1. **Tu confirmas visualmente** que la sesion esta iniciada (avatar arriba a la derecha). Si no, inicias sesion, escaneas el QR o resuelves el deslizador. El agente no pide contrasenas ni las introduce, y no debe sustituir este paso por `boss status`.
2. Sincroniza y verifica:

```
boss --cdp-url http://localhost:9222 login --cdp
agent-reach doctor      # la linea de boss debe mostrar la cookie wt2 del navegador
```

- Cualquier proceso local con acceso al puerto 9222 controla por completo ese Chrome: usa siempre `--remote-debugging-address=127.0.0.1`, nunca lo expongas a la red local o a internet.
- Usa un perfil independiente y persistente; no lo borres ni lo recrees cada vez, y no lo cambies por tu Chrome de diario.
- Cierra el Chrome dedicado cuando no lo uses.
- Las busquedas necesitan `--browser-source existing-browser --cdp-url http://localhost:9222`.
- No funciona en servidores ni entornos sin escritorio.
- El instalador fija un commit concreto de boss-agent-cli (`4c991b77086a203173bf08a4cb64a23af6514fe6`) en lugar de una rama movil. Cuando el upstream publique una version estable se pasara a una restriccion por version.

10. Uso diario

No hace falta memorizar comandos: el agente lee el `SKILL.md` y sabe que herramienta llamar. Estos son los comandos que ejecuta por debajo.

Peticion	Comando
Leer una web	`curl -s "https://r.jina.ai/URL"`
Que es un repo de GitHub	`gh repo view owner/repo`
Buscar repos	`gh search repos "LLM framework"`

Peticion	Comando
Subtitulos de un video de YouTube	`yt-dlp --dump-json URL`
Buscar en Bilibili	`bili search "query" --type video`
Subtitulos de Bilibili	`opencli bilibili subtitle BVxxx`
Busqueda semantica en la web	`mcporter call exa.web_search_exa query="..." numResults=5`
Buscar en Twitter	`twitter search "query" -n 10` (con TWITTER_AUTH_TOKEN y TWITTER_CT0)
Leer un tuit	`twitter tweet URL` (con las mismas variables)
Buscar en Reddit	`opencli reddit search "query" -f yaml` o `rdt read POST_ID`
Xiaohongshu	`opencli xiaohongshu search "query" -f yaml`
LinkedIn	`mcporter call linkedin.get_person_profile linkedin_username="..."`
RSS	`feedparser` (Python)
Transcribir podcast Xiaoyuzhou	`bash ~/.agent-reach/tools/xiaoyuzhou/transcribe.sh URL`

Si twitter-cli no esta disponible, se puede buscar contenido de Twitter con Exa:

```
mcporter call exa.web_search_exa query="site:x.com terminos de busqueda" numResults=5
```

En plataformas con varios backends, el activo es el que indica `agent-reach doctor --json` en `active_backend`.

11. Actualizar

Con tu agente:

```
Actualiza Agent Reach: https://raw.githubusercontent.com/Panniantong/agent-reach/main/docs/update.md
```

De forma manual, en este orden:

Paso 1. Comprobar la version

```
agent-reach check-update
```

Si dice que ya estas en la ultima version, salta al paso 5.

Paso 2. Actualizar el paquete

```
pip install --upgrade https://github.com/Panniantong/agent-reach/archive/main.zip
```

Si pip se queja de `externally-managed-environment`, instalaste con pipx o con un venv. Usa el comando que corresponda:

```
pipx install --force https://github.com/Panniantong/agent-reach/archive/main.zip
# o bien activa antes ~/.agent-reach-venv
```

Paso 3. Refrescar las herramientas upstream

Solo se actualiza lo que ya esta instalado; no se instala nada que no hayas pedido (con la unica excepcion de OpenCLI en escritorio, ver paso 4).

```
which twitter >/dev/null 2>&1 && { pipx upgrade twitter-cli 2>/dev/null \
  || uv tool upgrade twitter-cli 2>/dev/null; }
which bili >/dev/null 2>&1 && { pipx upgrade bilibili-cli 2>/dev/null \
  || uv tool upgrade bilibili-cli 2>/dev/null; }
which xhs >/dev/null 2>&1 && { pipx upgrade xiaohongshu-cli 2>/dev/null \
  || uv tool upgrade xiaohongshu-cli 2>/dev/null; }
which yt-dlp >/dev/null 2>&1 && { pipx install --force 'yt-dlp[default]' 2>/dev/null \
  || uv tool install --force 'yt-dlp[default]' 2>/dev/null \
  || python -m pip install -U 'yt-dlp[default]' 2>/dev/null; }

# rdt-cli va fijado a un commit de git (PyPI va por detras)
which rdt >/dev/null 2>&1 && pipx install --force \
  'git+https://github.com/public-clis/rdt-cli.git@5e4fb3720d5c174e976cd425ccc3b879d52cac66' 2>/dev/null

# basadas en npm
which mcporter >/dev/null 2>&1 && npm update -g mcporter 2>/dev/null
which opencli  >/dev/null 2>&1 && npm update -g @jackwener/opencli 2>/dev/null
```

Paso 4. OpenCLI para usuarios de escritorio que aun no lo tienen

Desde la version 1.5.0, OpenCLI es el backend preferido para Xiaohongshu y Reddit, y anade subtitulos de Bilibili. El update lo ofrece una sola vez. Para Xiaohongshu solo puede usar una sesion de Chrome existente que controles tu; la actualizacion nunca inicia sesion ni lee cookies. Si aceptas:

```
agent-reach install --system --channels opencli
```

Despues hay que dar el clic manual en la extension (9.5). Si dices que no, todo sigue funcionando con los backends existentes. Sin sesion previa, no se hace login automatico: se recurre a Cookie-Editor con MCP o herramienta heredada.

Paso 5. Coexistencia: no desinstales herramientas antiguas

Los backends retirados (por ejemplo, yt-dlp para Bilibili, o xhs-cli que ya no se instala por defecto) siguen como respaldo mientras funcionen. Agent Reach enruta alrededor de ellos automaticamente. Quitarlos es decision tuya, no del agente.

Paso 6. Verificar

```
agent-reach version
agent-reach doctor
```

Paso 7. Informe final

Al terminar deberias saber: la version actual, cuantos canales hay disponibles y que backend usa cada plataforma multi-backend, si queda alguna accion tuya pendiente (clic en extension de Chrome o exportar cookies) y los cambios de la actualizacion (las notas de version las muestra `check-update`).

12. Monitorizacion diaria (OpenClaw)

```
agent-reach watch          # comprobacion rapida de estado + versiones (pensada para tareas programadas)
agent-reach check-update
```

Si trabajas en **OpenClaw**, al terminar la instalacion el agente te ofrece crear una tarea diaria (cron con `sessionTarget: "isolated"` y `delivery: "announce"`) que ejecuta `agent-reach watch`. La tarea funciona asi:

- Si la salida contiene la frase china que indica que todo esta normal, termina en silencio.
- Si hay problemas (marcas de error o aviso) o una version nueva, te envia el informe completo con una propuesta de arreglo.
- Si hay version nueva, te pregunta si quieres actualizar (con la frase de actualizacion de la seccion 11).

13. Resolucion de problemas

Sintoma	Causa	Solucion
`externally-managed-environment`	Proteccion PEP 668 de tu sistema	Usa pipx o un entorno virtual (6.1 y 6.2)
`python3` abre la Microsoft Store (Windows)	Alias de la Store, no es Python real	Usa `py -3` o el `python.exe` real (6.3)
Twitter: error al ejecutar o `Missing credentials`	Faltan TWITTER_AUTH_TOKEN y TWITTER_CT0 en el proceso	Exportalas. Para comprobar: `twitter check`
Twitter no llega a x.com	Tu red necesita proxy	Exporta HTTP_PROXY y HTTPS_PROXY (9.3), o usa un proxy global
Twitter sigue sin funcionar	twitter-cli no disponible	Busca con Exa, o usa bird si lo tienes
Xueqiu: `HTTP Error 400`	La API exige cookies de login	`agent-reach configure --from-browser chrome --platform xueqiu`
Boss: `AUTH_EXPIRED`	El Chrome dedicado no tiene sesion	Ver detalle mas abajo
Reddit no funciona	No hay ruta anonima	Necesitas sesion: OpenCLI o `rdt login`
Bilibili con yt-dlp da 412	Bloqueo de Bilibili	Usa bili-cli (ya es el backend por defecto)
Instagram error 429 o de login	Limite de peticiones o sesion caducada	Vuelve a iniciar sesion en Chrome y baja la frecuencia
"fetch failed"	Red o proxy	Configura proxy (9.3) y revisa troubleshooting.md del repo

Twitter/X: twitter-cli falla al conectar

Causa: twitter-cli necesita las variables `TWITTER_AUTH_TOKEN` y `TWITTER_CT0`. Lo que guarda `agent-reach configure twitter-cookies` solo sirve para que doctor compruebe si la configuracion esta completa; doctor no ejecuta la autenticacion upstream ni define variables en tu shell. Si tu red necesita proxy para llegar a x.com, tambien hay que configurarlo.

Solucion 1: variables de entorno y proxy

```
export TWITTER_AUTH_TOKEN="..."
export TWITTER_CT0="..."
export HTTP_PROXY="http://user:pass@host:port"
export HTTPS_PROXY="http://user:pass@host:port"
twitter search "test" -n 1
```

Solucion 2: proxy global

Deja que una herramienta de proxy gestione todo el trafico, incluido el de twitter-cli. En macOS, ClashX o Surge con "modo mejorado"; en Linux, proxychains o tun2socks:

```
proxychains twitter search "test" -n 1
```

Solucion 3: usar Exa en lugar de twitter-cli

```
mcporter call exa.web_search_exa query="site:x.com terminos" numResults=5
```

Solucion 4: comprobar la autenticacion

```
twitter check
```

Si devuelve "Missing credentials", define TWITTER_AUTH_TOKEN y TWITTER_CT0 en el entorno del proceso que ejecuta el comando. Como respaldo, si tienes bird CLI (`npm install -g @steipete/bird`), tambien funciona y Agent Reach lo detecta solo.

Xueqiu: la API devuelve 400

Sintoma: `agent-reach doctor` muestra un aviso en Xueqiu con `HTTP Error 400`. Causa: la API exige cookies de login. Solucion: inicia sesion en xueqiu.com en Chrome y ejecuta:

```
agent-reach configure --from-browser chrome --platform xueqiu
```

Vuelve a ejecutar doctor para confirmar. Cuando la cookie caduque, repite el comando.

Boss Zhipin: boss status dice que hay sesion pero la busqueda da AUTH_EXPIRED

Sintoma: `boss status` (o `status --live`) devuelve `logged_in: true`, pero la busqueda falla enseguida con `{"code": "AUTH_EXPIRED", "message": ...}`. El Chrome dedicado puede estar ademas en una URL con `_security_check`, que parece un deslizador antibots.

Causa: Boss guarda la sesion en **dos sitios distintos** que autentican canales diferentes:

Almacen	Quien lo usa
`~/.boss-agent/auth/session.enc`	boss status / status --live y las operaciones de bajo riesgo por httpx (detail, cities, job_card_httpx). La busqueda CDP tambien lo lee: si no puede, falla como no logueado. Pero con el contexto de Chrome real, sus cookies nunca surten efecto.
`~/.boss-chrome-profile` (cookies del navegador)	En modo estricto existing-browser (CDP), las operaciones de riesgo alto (search, greet...) usan las credenciales del navegador.

`boss status` solo valida el archivo local. Si ese archivo guarda credenciales antiguas y el perfil de Chrome no esta logueado, sigue diciendo `logged_in: true`, lo cual no prueba que la sesion sea valida. Ademas, la pagina `_security_check` es un reto antibots que **aparece incluso con sesion iniciada**, asi que no sirve para juzgar el estado.

> **Importante:** No borres ninguno de los dos almacenes. Si falta session.enc, la busqueda CDP falla antes de conectar con el navegador. Para refrescarlo, ejecuta `login --cdp` en lugar de borrarlo a mano.

Orden para decidir:

1. `AUTH_EXPIRED` es la verdad: significa que el navegador no tiene sesion, diga lo que diga boss status.
2. La linea de boss en `agent-reach doctor` comprueba directamente si hay cookie `wt2` en el navegador; usala como referencia.
3. `boss status` solo como dato orientativo; la URL de la pagina nunca es criterio.

Solucion: confirma a ojo y haz login manual en zhipin.com en la ventana de Chrome dedicada, y sincroniza:

```
boss --cdp-url http://localhost:9222 login --cdp
agent-reach doctor      # la linea de boss debe decir que hay cookie de login (wt2) en el navegador
```

Tras lanzar el Chrome dedicado, el primer paso es siempre que el usuario confirme visualmente el estado de la sesion.

14. Seguridad y privacidad

Medida	Descripcion
Credenciales locales	Cookies y tokens solo en `~/.agent-reach/config.yaml` con permisos 600 (solo el propietario). No se suben ni se comparten.
Seguro por defecto	`agent-reach install` no modifica el sistema salvo que pases `--system`.
Codigo abierto	Todo es auditable, incluidas las herramientas de las que depende.
Dry run	`agent-reach install --dry-run` previsualiza todas las operaciones sin cambiar nada.
Arquitectura modular	Si no confias en un componente, sustituyes su archivo de canal sin afectar al resto.

> **Riesgo de suspension de cuenta:** Los plataformas que usan cookies o sesion de navegador (Twitter, Xiaohongshu, Reddit, Facebook, Instagram, Boss Zhipin) **pueden detectar llamadas no propias de un navegador y restringir o suspender la cuenta**. Usa siempre una **cuenta secundaria** y nunca la principal. Ademas, una cookie equivale a acceso completo a la cuenta: con una cuenta secundaria limitas el dano si se filtra.

Nota sobre archivos: el README indica `config.yaml` y el documento de instalacion menciona `config.json` en un ejemplo. La carpeta es la misma (`~/.agent-reach/`); si dudas, listala y comprueba cual existe.

15. Desinstalar

```
agent-reach uninstall                # borra ~/.agent-reach/ (tokens y cookies incluidos), skills y config MCP en m
agent-reach uninstall --dry-run      # solo previsualiza
agent-reach uninstall --keep-config  # borra solo los archivos de skill y conserva los tokens (util para reinstalar
```

Para quitar el paquete Python en si:

```
pip uninstall agent-reach
```

(Si lo instalaste con pipx, el equivalente es `pipx uninstall agent-reach`; el repositorio solo documenta el comando de pip.)

16. Referencia rapida de comandos

Comando	Que hace
`agent-reach install --env=auto`	Comprobacion de solo lectura de dependencias y canales (por defecto)
`agent-reach install --env=auto --system`	Instala y configura las herramientas externas principales
`agent-reach install --env=auto --system --channels=twitter,xiaohongshu`	Instala los canales opcionales aprobados
`agent-reach install --env=local --system --channels=boss`	Instala el backend de escritorio de Boss Zhipin (CDP estricto)
`agent-reach install --env=auto --system --channels=all`	Instala todo, tras aprobacion explicita
`agent-reach install --env=auto --safe`	Alias de compatibilidad de la comprobacion segura
`agent-reach install --env=auto --dry-run`	Previsualiza lo que se haria
`agent-reach doctor`	Estado de cada canal y backend activo
`agent-reach doctor --json`	Lo mismo en formato para maquinas (active_backend)
`agent-reach watch`	Comprobacion rapida de salud y versiones (para tareas programadas)

Comando	Que hace
`agent-reach check-update`	Comprueba si hay versiones nuevas
`agent-reach version`	Muestra la version instalada
`agent-reach skill --install`	Refresca los archivos de la skill incluida
`agent-reach configure twitter-cookies`	Guarda las cookies de Twitter (entrada oculta) para que doctor las compruebe
`agent-reach configure proxy`	Guarda la direccion del proxy (entrada oculta); no lo activa automaticamente
`agent-reach configure groq-key`	Guarda la clave de Groq para transcribir podcasts
`agent-reach configure xhs-cookies`	Guarda cookies exportadas de xiaohongshu.com (no las inyecta en OpenCLI ni Chrome)
`agent-reach configure --from-browser chrome --platform xueqiu`	Lee solo las cookies de Xueqiu desde Chrome
`agent-reach uninstall [--dry-run | --keep-config]`	Desinstala configuracion, skills y config MCP

17. Enlaces y creditos

Documentos del repositorio

- Repositorio: github.com/Panniantong/Agent-Reach
- Instalacion: raw.githubusercontent.com/Panniantong/agent-reach/main/docs/install.md
- Actualizacion: raw.githubusercontent.com/Panniantong/agent-reach/main/docs/update.md
- Problemas frecuentes: raw.githubusercontent.com/Panniantong/Agent-Reach/main/docs/troubleshooting.md
- README en ingles: github.com/Panniantong/Agent-Reach/blob/main/docs/README_en.md (tambien hay en japones y coreano)
- Otros archivos en la raiz: CHANGELOG.md, CONTRIBUTING.md, SECURITY.md, llms.txt, CLAUDE.md, .env.example, pyproject.toml, constraints.txt, test.sh
- Carpetas: agent_reach, config, docs, scripts, tests, .github/workflows, .openteams/specs
- Bugs y peticiones de funcionalidades: pestana Issues del repositorio

Herramientas upstream que utiliza

OpenCLI (github.com/jackwener/opencli), twitter-cli y rdt-cli y bili-cli (github.com/public-clis), xiaohongshu-mcp (github.com/xpzouying/xiaohongshu-mcp), xhs-cli (github.com/jackwener/xiaohongshu-cli), yt-dlp, Jina Reader, Exa (exa.ai), mcporter (github.com/nicobailon/mcporter), feedparser y mcp-server-linkedin (github.com/stickerdaniel/linkedin-mcp-server).

Autor y contacto

Contacto del autor segun el README: pnt01@foxmail.com y @Neo_Reidlab en X. Para colaboraciones el README remite a WeChat; para bugs, GitHub Issues.

Patrocinadores mencionados en el README

BrowserAct, Tencent Cloud (OpenClaw en Lighthouse), CoreClaw y AstraFlow de UCloud. Son anuncios de patrocinadores del repositorio, no requisitos para usar Agent Reach.